Name : Jenil Ambawala
Roll No : 27
Division : A

1. Data Acquisition and Pre-processing

- **Read the provided `.csv` dataset containing Riya's 15-day store activity.**
- **Perform data cleaning: Check for missing or inconsistent entries.**
- **Apply pre-processing using Pandas and NumPy.**

```
PS D:\MSC-CA sem-3 Tej\assiment> python casestudy.py
Original Data:
          Day        Date  Footfall    Sales  Ad_Spend  Insta_Mentions  Whatsapp_Inquiries Deal_Offered
0   Wednesday  2025-01-01      77.0   9077.0    1927.0             NaN                23.0          Yes
1    Thursday  2025-01-02      23.0   4700.0    1383.0             NaN                 7.0           No
2      Friday  2025-01-03      87.0   9379.0    1860.0            10.0                23.0          Yes
3    Saturday  2025-01-04     140.0  11145.0     402.0             9.0                35.0           No
4      Sunday  2025-01-05      76.0   8587.0     558.0            10.0                29.0           No
..        ...         ...       ...      ...       ...             ...                 ...          ...
95     Sunday  2025-04-06      32.0   4330.0    1659.0             4.0                 NaN           No
96     Monday  2025-04-07     122.0   8798.0    2101.0             6.0                21.0          Yes
97    Tuesday  2025-04-08     147.0  10276.0    1804.0             8.0                17.0          NaN
98  Wednesday  2025-04-09      40.0   5423.0    2063.0             9.0                11.0           No
99   Thursday  2025-04-10      40.0   6417.0       NaN            10.0                18.0           No

[100 rows x 8 columns]
Missing Values:
 Day                  0
Date                  0
Footfall              3
Sales                 1
Ad_Spend              4
Insta_Mentions        5
Whatsapp_Inquiries    1
Deal_Offered          6
dtype: int64
```

```
[100 rows x 8 columns]
          Day        Date  Footfall    Sales     Ad_Spend  Insta_Mentions  Whatsapp_Inquiries Deal_Offered  Deal_Encode
0   Wednesday  2025-01-01      77.0   9077.0  1927.000000        4.884211           23.000000          Yes            1
1    Thursday  2025-01-02      23.0   4700.0  1383.000000        4.884211            7.000000           No            0
2      Friday  2025-01-03      87.0   9379.0  1860.000000       10.000000           23.000000          Yes            1
3    Saturday  2025-01-04     140.0  11145.0   402.000000        9.000000           35.000000           No            0
4      Sunday  2025-01-05      76.0   8587.0   558.000000       10.000000           29.000000           No            0
..        ...         ...       ...      ...          ...             ...                 ...          ...          ...
95     Sunday  2025-04-06      32.0   4330.0  1659.000000        4.000000           19.040404           No            0
96     Monday  2025-04-07     122.0   8798.0  2101.000000        6.000000           21.000000          Yes            1
97    Tuesday  2025-04-08     147.0  10276.0  1804.000000        8.000000           17.000000           No            0
98  Wednesday  2025-04-09      40.0   5423.0  2063.000000        9.000000           11.000000           No            0
99   Thursday  2025-04-10      40.0   6417.0  1568.916667       10.000000           18.000000           No            0

[100 rows x 9 columns]
Missing Values:
 Day                  0
Date                  0
Footfall              0
Sales                 0
Ad_Spend              0
Insta_Mentions        0
Whatsapp_Inquiries    0
Deal_Offered          0
Deal_Encode           0
dtype: int64
```

2. Data Transformation

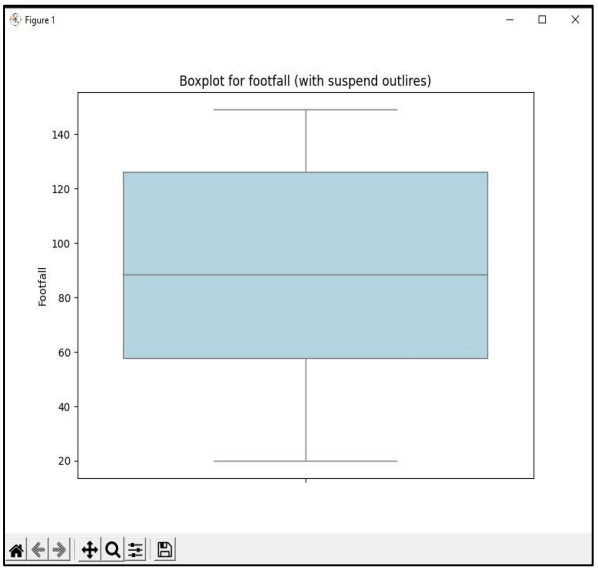
- **Normalize numerical columns (like sales, ad spend, inquiries) using Min-Max or Z-score normalization.**
- **Convert categorical data (like Day or Deal Offered) into numerical format for modeling.**

```
    Sales_Normalized
0           0.720125
1           0.127757
2           0.760996
3           1.000000
4           0.653810
..               ...
95          0.077683
96          0.682366
97          0.882393
98          0.225606
99          0.360130

[100 rows x 1 columns]
    Day        Date  Footfall    Sales     Ad_Spend  Insta_Mentions  Whatsapp_Inquiries Deal_Offered  Deal_Encode  Sales_Normalized
0     3  2025-01-01      77.0   9077.0  1927.000000        4.884211           23.000000          Yes            1         0.720125
1     4  2025-01-02      23.0   4700.0  1383.000000        4.884211            7.000000           No            0         0.127757
2     5  2025-01-03      87.0   9379.0  1860.000000       10.000000           23.000000          Yes            1         0.760996
3     6  2025-01-04     140.0  11145.0   402.000000        9.000000           35.000000           No            0         1.000000
4     0  2025-01-05      76.0   8587.0   558.000000       10.000000           29.000000           No            0         0.653810
..  ...         ...       ...      ...          ...             ...                 ...          ...          ...              ...
95    0  2025-04-06      32.0   4330.0  1659.000000        4.000000           19.040404           No            0         0.077683
96    1  2025-04-07     122.0   8798.0  2101.000000        6.000000           21.000000          Yes            1         0.682366
97    2  2025-04-08     147.0  10276.0  1804.000000        8.000000           17.000000           No            0         0.882393
98    3  2025-04-09      40.0   5423.0  2063.000000        9.000000           11.000000           No            0         0.225606
99    4  2025-04-10      40.0   6417.0  1568.916667       10.000000           18.000000           No            0         0.360130
```

3. Exploratory Data Analysis (EDA)

- Plot summary statistics: mean, median, mode, etc.
- Identify any outliers in the data (e.g., a sudden spike in inquiries or footfall).
- Check correlations between:
    - Influencer mentions and sales
    - Ad spend and footfall
    - WhatsApp inquiries and actual purchases

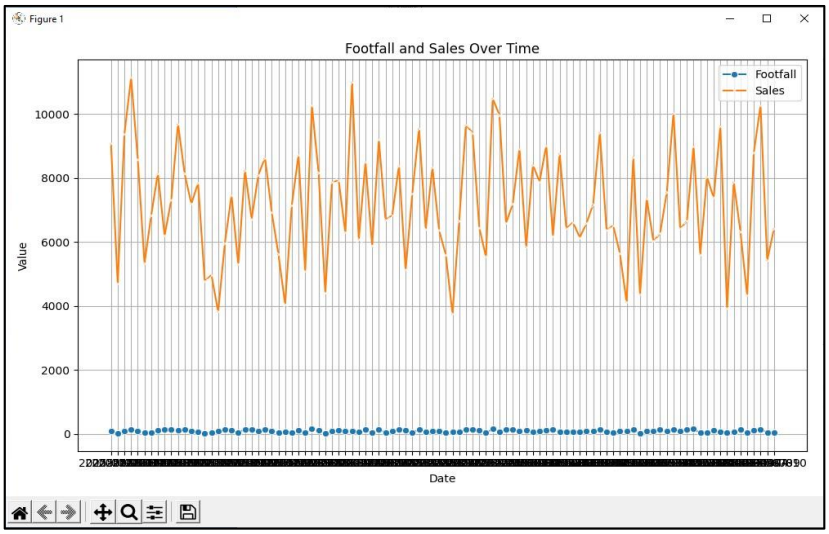
```
 Desc Stattistic
              Day    Footfall         Sales     Ad_Spend  Insta_Mentions  Whatsapp_Inquiries  Deal_Encode  Sales_Normalized
count  100.000000  100.000000    100.000000   100.000000      100.000000          100.000000   100.000000        100.000000
mean     3.010000   88.525773   7197.242424  1568.916667        4.884211           19.040404     0.460000          0.465725
std      1.992384   38.301856   1757.547362   606.034047        3.177774           11.965370     0.500908          0.237860
min      0.000000   20.000000   3756.000000   331.000000        0.000000            0.000000     0.000000          0.000000
25%      1.000000   57.750000   6059.750000  1180.750000        2.000000            8.750000     0.000000          0.311781
50%      3.000000   88.525773   7048.000000  1662.000000        5.000000           19.000000     0.000000          0.445527
75%      5.000000  126.250000   8521.000000  2059.250000        8.000000           29.000000     1.000000          0.644878
max      6.000000  149.000000  11145.000000  2495.000000       10.000000           40.000000     1.000000          1.000000
correlations between Insta_Mentions & Sales 0.3344462997564363
correlations between Ad_Spend & Footfall 0.005962643092664303
correlations between Whatsapp_Inquiries & Sales 0.49989970158931946
```



4. Data Visualization –

Use Matplotlib and Seaborn to:

- **Plot footfall and sales over time**
- **Visualize relationships using scatterplots or pair plots**
- **Show distribution of daily footfall/sales using histograms or boxplots**



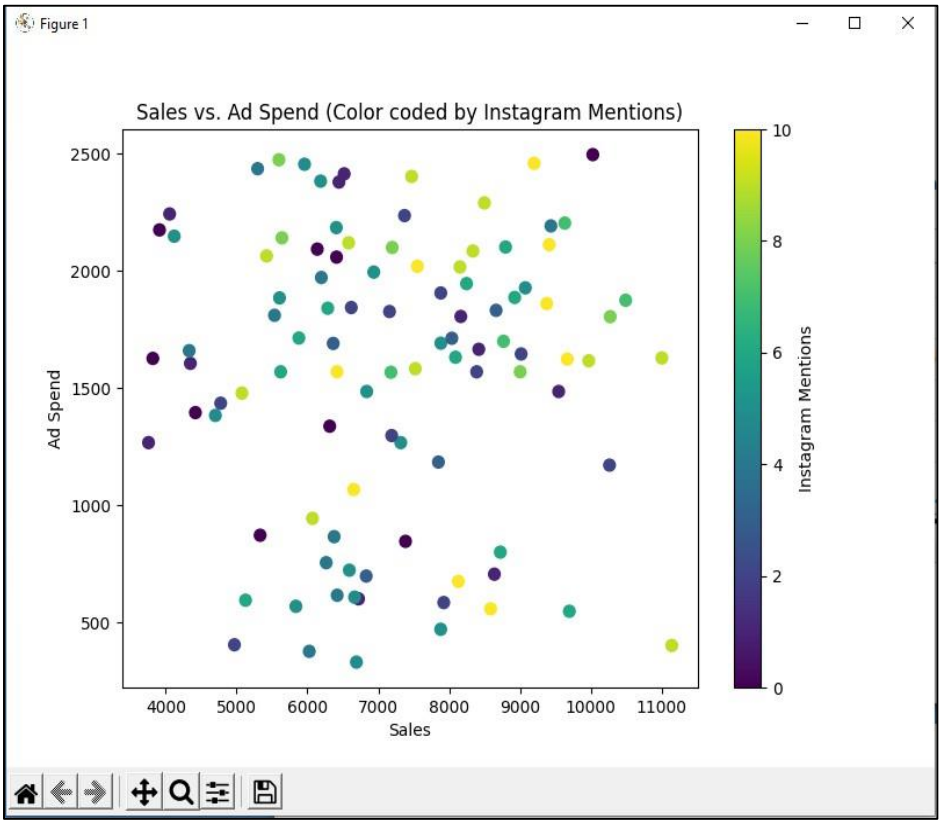


Figure 1
Sales vs. Ad Spend (Color coded by Instagram Mentions)
Ad Spend
Sales
Instagram Mentions
2500
2000
1500
1000
500
4000
5000
6000
7000
8000
9000
10000
11000
10
8
6
4
2
0

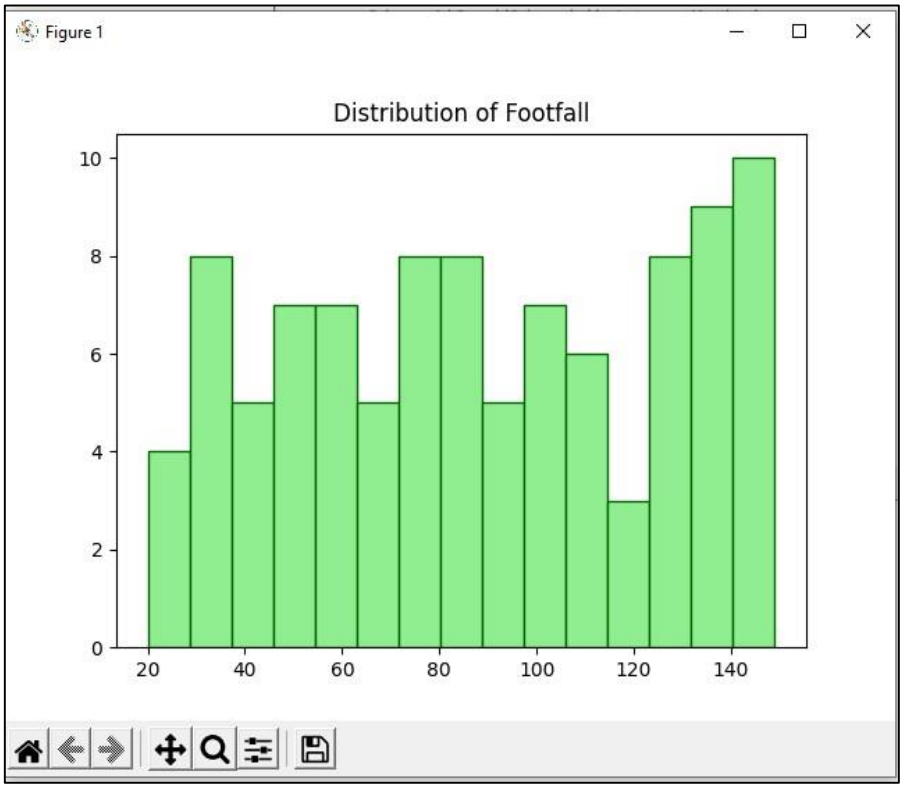
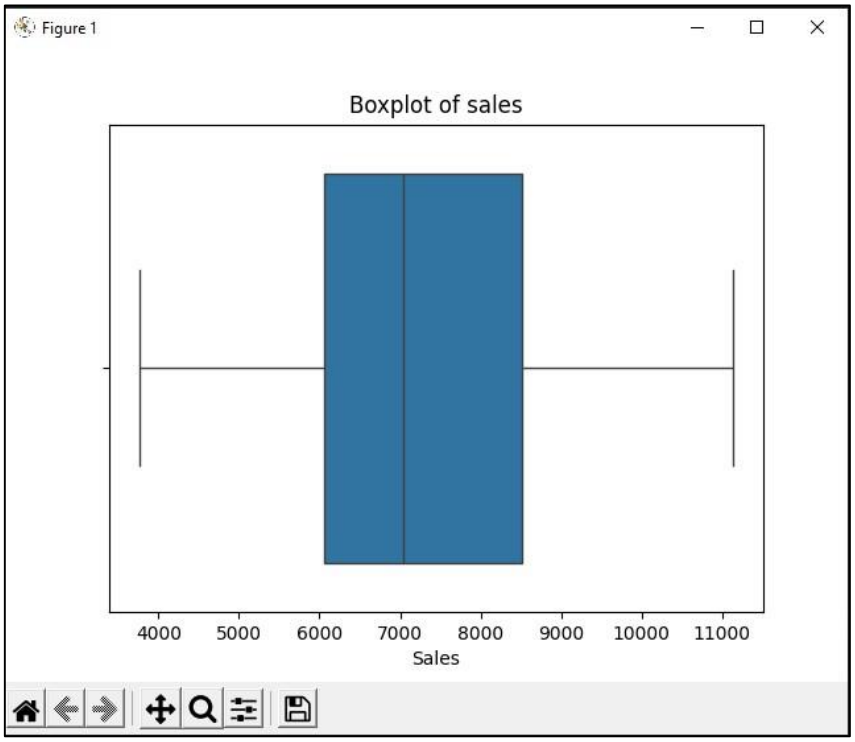


Figure 1
Distribution of Footfall
10
8
6
4
2
0
20
40
60
80
100
120
140



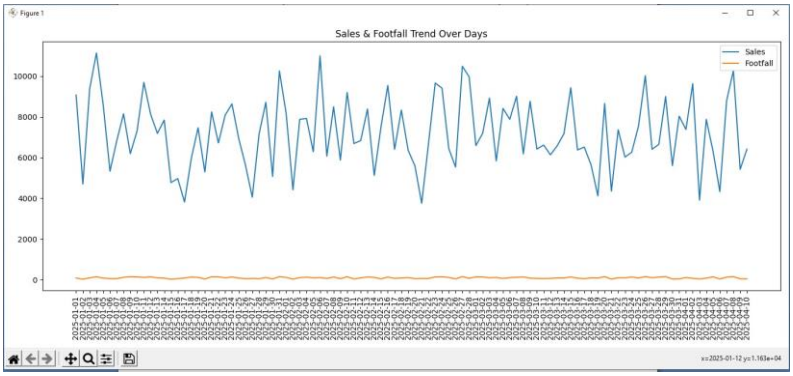
5. Data Modeling and Analysis

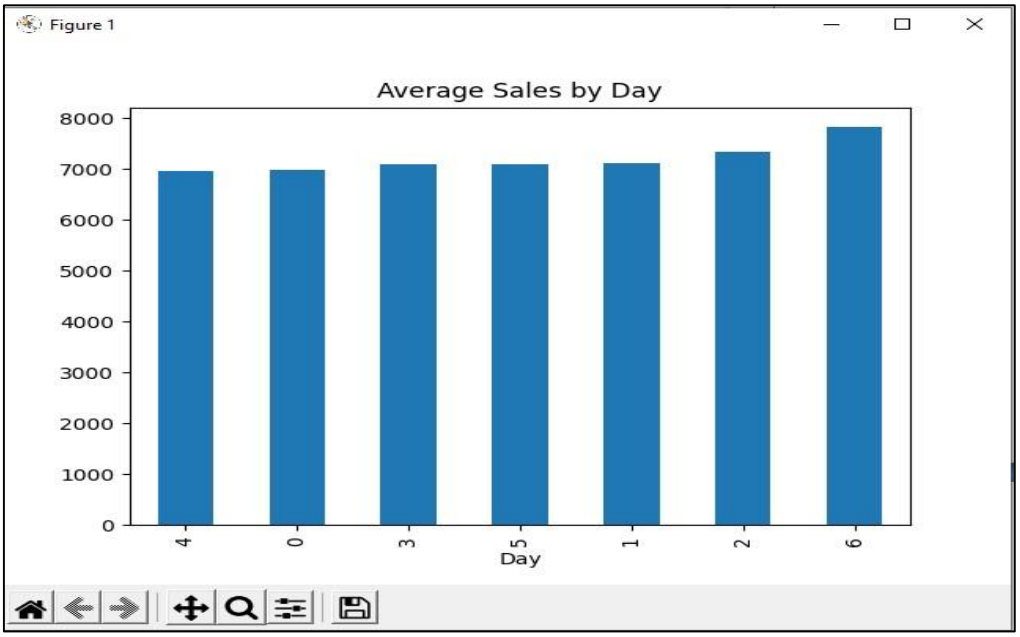
- **Build a Linear Regression model to predict sales using independent variables (ad spend, footfall, inquiries, influencer posts).**
- **Build a Logistic Regression model to predict the probability of 'High Sales Day' (1 if sales > ₹5000, 0 otherwise).**
- **Use summary outputs to explain what variables have the most influence.**

```
Linear Regression Coefficients:
Ad_Spend: -0.1412
Footfall: 23.8703
Insta_Mentions: 165.8033
Whatsapp_Inquiries: 71.9721
```

6. Bonus – Time Series Element

- **Try plotting sales and footfall over days.**
- **Discuss whether there's a weekly pattern or trend (e.g., higher sales on weekends).**





Code:-

```
import numpy as np import pandas as pd from sklearn.preprocessing import LabelEncoder,
MinMaxScaler from sklearn.linear_model
import LinearRegression, LogisticRegression
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv('threadora_boutique_data.csv') df = pd.DataFrame(data) print("Original Data:\n",
df)

# missing value print("Missing Values:\n", df.isnull().sum())
 df['Ad_Spend'].fillna(df['Ad_Spend'].mean(),
                       inplace=True) print(df)
 df['Footfall'].fillna(df['Footfall'].mean(),
                       inplace=True) print(df)
 df['Sales'].fillna(df['Sales'].mean(), inplace=True) print(df)
 df['Insta_Mentions'].fillna(df['Insta_Mentions'].mean(),
```

```
                    inplace=True) print(df)
 df['Whatsapp_Inquiries'].fillna(df['Whatsapp_Inquiries'].mean(), inplace=True) print(df)
 df['Deal_Offered'].fillna(df['Deal_Offered'].mode()[0],
                   inplace=True) print(df)
 dealoffer = LabelEncoder() df['Deal_Encode'] = dealoffer.fit_transform(df['Deal_Offered']) print(df)
print("Missing Values:\n", df.isnull().sum())

# # Min-Max Normalization for
Marks scaler = MinMaxScaler() df['Sales_Normalized'] = scaler.fit_transform(df[['Sales']])
print("\nTransformed Data:\n", df[['Sales_Normalized']])
 df['Day'] = df['Day'].replace({'Sunday': 0 ,'Monday': 1,'Tuesday': 2,'Wednesday': 3,'Thursday':
4,'Friday': 5,'Saturday': 6}) print(df)
 print("\n Desc Stattistic") print(df.describe())
 plt.figure(figsize=(16,20))
sns.boxplot(y=df['Footfall'], color='lightblue') plt.title("Boxplot for footfall (with suspend outlires)")
plt.show()

Q1 = df['Footfall'].quantile(0.25)
Q3 = df['Footfall'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds lower_
bound = Q1 - 1.5 * IQR upper_bound = Q3 + 1.5 * IQR

# Identify outliers
 outliers = df[(df['Footfall'] < lower_bound) | (df['Footfall'] > upper_bound)]

corr = df['Insta_Mentions'].corr(df['Sales']) print("correlations between Insta_Mentions &
Sales",corr)
 corr = df['Ad_Spend'].corr(df['Footfall']) print("correlations between Ad_Spend & Footfall",corr)
 corr = df['Whatsapp_Inquiries'].corr(df['Sales']) print("correlations between Whatsapp_Inquiries &
Sales",corr)

# ques 4 plt.figure(figsize=(10, 6)) sns.lineplot(x='Date', y='Footfall', data=df, label='Footfall',
marker='o') sns.lineplot(x='Date', y='Sales', data=df, label='Sales', marker='x') plt.title('Footfall and
Sales Over Time') plt.xlabel('Date') plt.ylabel('Value') plt.legend() plt.grid(True) plt.tight_layout()
plt.show()

df = pd.DataFrame(data)
 x = df['Sales'] y = df['Ad_Spend'] z = df['Insta_Mentions']
 plt.figure(figsize=(8, 6))   plt.scatter(x, y, c=z, cmap='viridis', s=50) plt.xlabel("Sales") plt.ylabel("Ad
Spend") plt.title("Sales vs. Ad Spend (Color coded by Instagram Mentions)")
plt.colorbar(label='Instagram Mentions')  plt.show()
df = pd.DataFrame(data) plt.hist(df['Footfall'], bins=15, color='lightgreen', edgecolor='darkgreen')
plt.title('Distribution of Footfall') plt.show()
 sns.boxplot(x=df['Sales']) plt.title('Boxplot of sales') plt.show()
```

```
# 5
# Build a Linear Regression model to predict sales using independent variables (ad # spend, footfall, inquiries, influencer posts)
X = df[['Ad_Spend', 'Footfall', 'Insta_Mentions', 'Whatsapp_Inquiries']] y = df['Sales']
model = LinearRegression() model.fit(X, y)
print("Linear Regression Coefficients:") for feature, coef in zip(X.columns, model.coef_):
  print(f"{feature}: {coef:.4f}")

# # - Build a Logistic Regression model to predict the probability of ‘High Sales Day’ (1 if
# sales > ₹5000, 0 otherwise).
df['High_Sales_Day'] = (df['Sales'] > 0.5).astype(int)
X = df[['Footfall', 'Ad_Spend', 'Whatsapp_Inquiries', 'Insta_Mentions']] y = df['High_Sales_Day']
log_model = LogisticRegression() log_model.fit(X, y)
print("Logistic Regression Coefficients:") for feature, coef in zip(X.columns, log_model.coef_[0]):
  print(f"{feature}: {coef:.4f}")
# 6
# Try plotting sales and footfall over days.
plt.figure(figsize=(14,6)) plt.plot(df['Date'], df['Sales'], label='Sales') plt.plot(df['Date'], df['Footfall'], label='Footfall') plt.xticks(rotation=90) plt.title('Sales & Footfall Trend Over Days') plt.legend()
plt.tight_layout() plt.show()

# Discuss whether there's a weekly pattern or trend (e.g., higher sales on weekends)
weekly_avg = df.groupby('Day')['Sales'].mean().sort_values() weekly_avg.plot(kind='bar', title="Average Sales by Day") plt.show()
```

Ques 2. Visualizing Cricketer Performance:

- Load a dataset containing cricket players' performance (e.g., runs scored, wickets taken, strike rate).
- Create a scatter plot to visualize the relationship between runs scored and strike rate.
- Use a bar chart to compare the total runs scored by top 5 players.
- Plot the progression of a player's runs across different matches using a line chart.

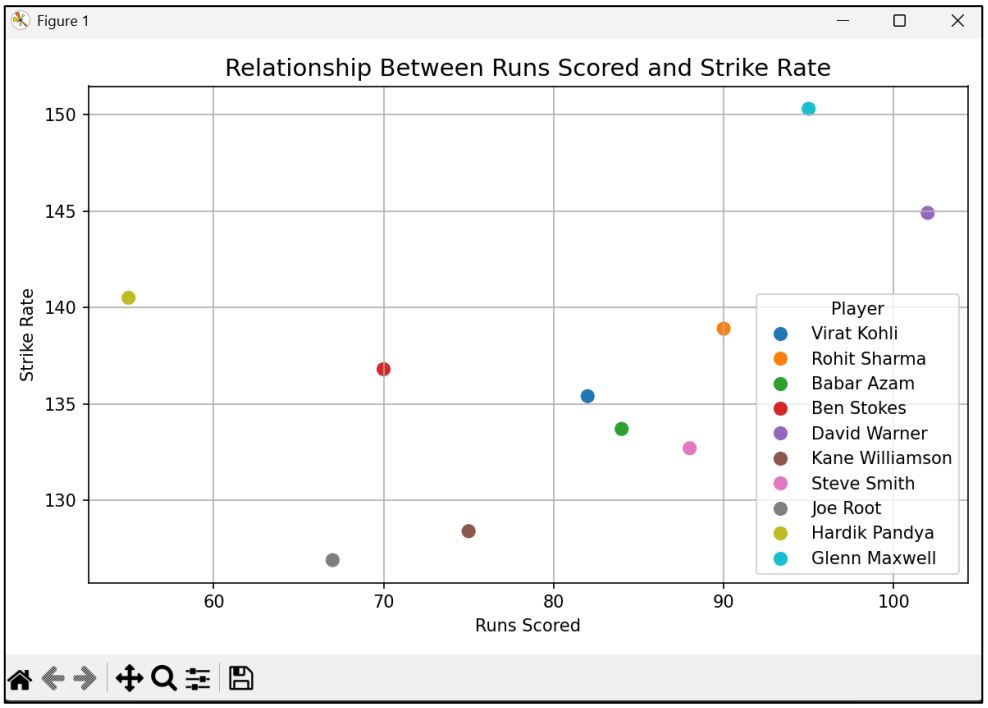
Code :-

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("cricketer_performance.csv")
print("Data Loaded Successfully!")
print(df.head())

plt.figure(figsize=(8, 5))
sns.scatterplot(x="Runs", y="StrikeRate", data=df, hue="Player", s=80)
plt.title("Relationship Between Runs Scored and Strike Rate", fontsize=14)
plt.xlabel("Runs Scored")
plt.ylabel("Strike Rate")
plt.grid(True)
plt.tight_layout()
plt.show()

total_runs = df.groupby("Player")["Runs"].sum().sort_values(ascending=False).reset_index()

plt.figure(figsize=(8, 5))
sns.barplot(x="Player", y="Runs", data=total_runs, palette="Blues_d")
plt.title("Total Runs Scored by Top 5 Players", fontsize=14)
plt.xlabel("Player")
plt.ylabel("Total Runs")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

player_name = "Virat Kohli"
player_data = df[df["Player"] == player_name]

plt.figure(figsize=(8, 5))
plt.plot(player_data["Match"], player_data["Runs"], marker="o", color="green", linewidth=2)
plt.title(f"Run Progression of {player_name} Across Matches", fontsize=14)
plt.xlabel("Match Number")
plt.ylabel("Runs Scored")
plt.grid(True)
plt.tight_layout()
```

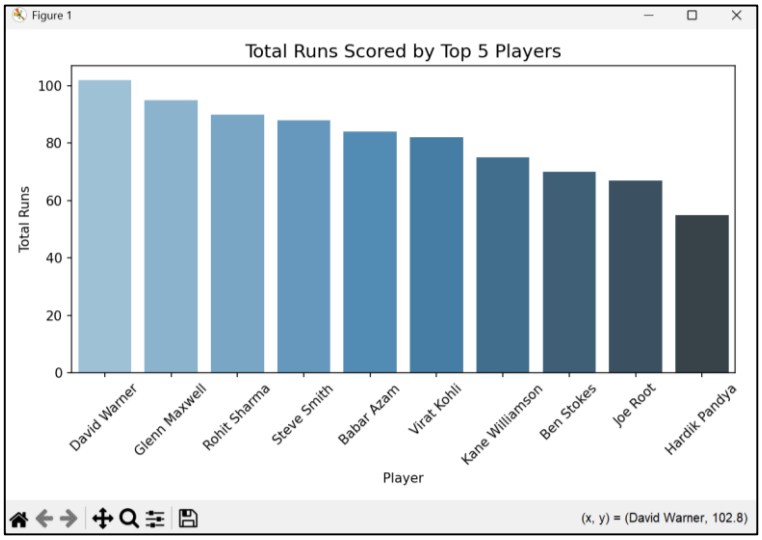
plt.show()

```
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV> python ques_2.py
Data Loaded Successfully!
         Player  Match  Runs  Wickets  StrikeRate
0   Virat Kohli      1    82        0       135.4
1  Rohit Sharma      1    90        0       138.9
2    Babar Azam      1    84        0       133.7
3    Ben Stokes      1    70        2       136.8
4  David Warner      1   102        0       144.9
```

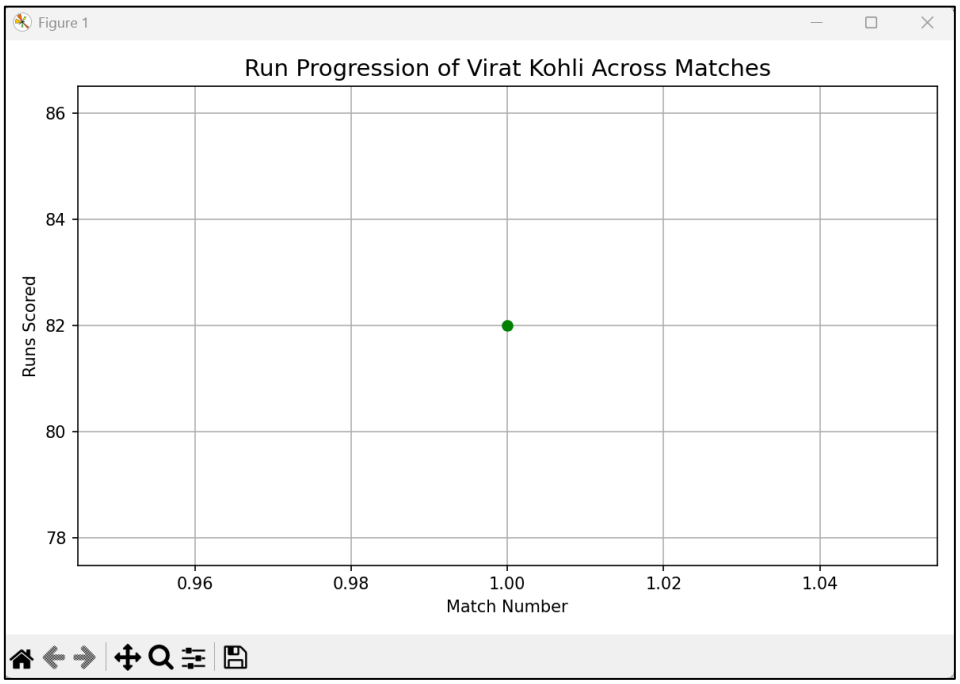
-Create a scatter plot to visualize the relationship between runs scored and strike rate.



-Use a bar chart to compare the total runs scored by top 5 players.



-Plot the progression of a player's runs across different matches using a line chart.



Ques 3. Tennis Match Analysis:
- Load a dataset of tennis matches and player performance (e.g., aces, double faults, win percentage).
- Create a bar chart comparing the number of aces and double faults for top 5 players.
- Plot a line chart to show the win percentage trend of a player over a season.
- Use Seaborn's pairplot() function to explore the relationship between different performance metrics.

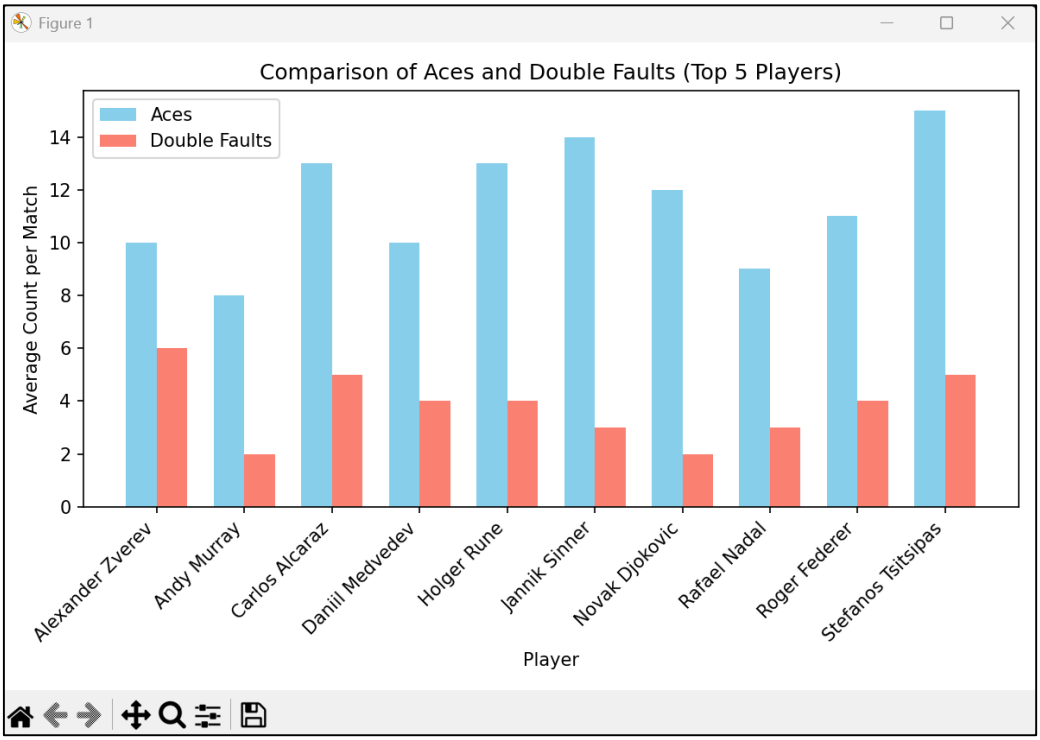
Code:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("tennis_data.csv")

print("Dataset Loaded Successfully!")
print(df.head())

avg_stats = df.groupby('Player')[['Aces', 'DoubleFaults']].mean().reset_index()

plt.figure(figsize=(8, 5))
x = np.arange(len(avg_stats['Player']))
width = 0.35
```

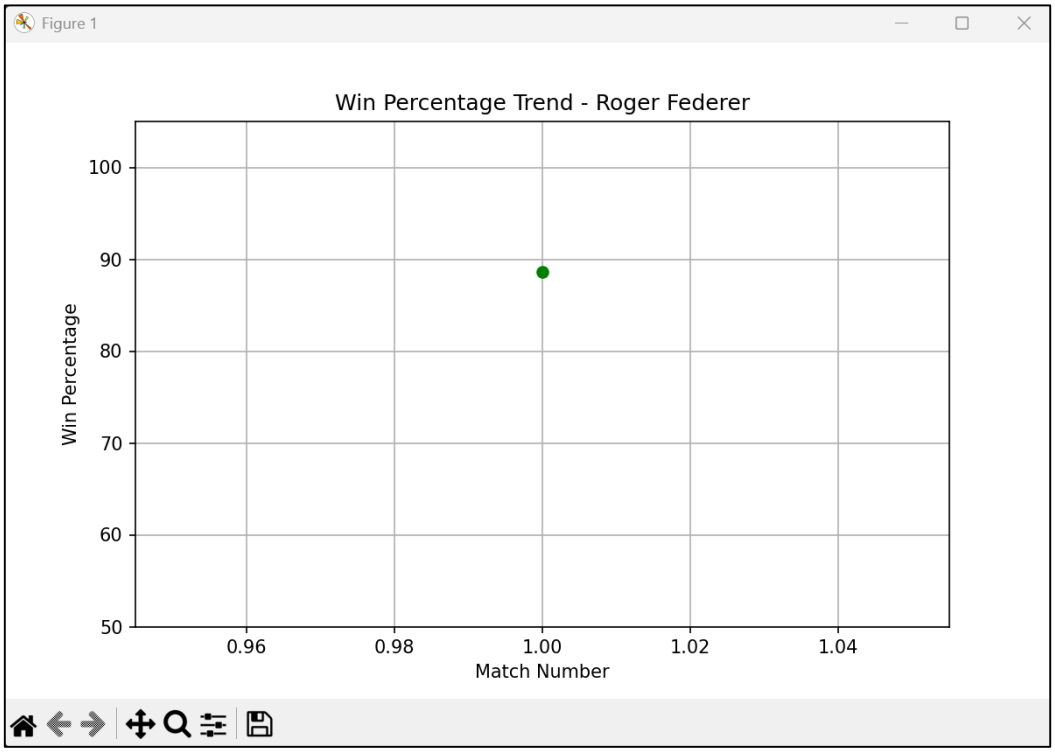
```
plt.bar(x - width/2, avg_stats['Aces'], width, label='Aces', color='skyblue')
plt.bar(x + width/2, avg_stats['DoubleFaults'], width, label='Double Faults', color='salmon')

plt.xticks(x, avg_stats['Player'], rotation=45, ha='right')
plt.xlabel('Player')
plt.ylabel('Average Count per Match')
plt.title('Comparison of Aces and Double Faults (Top 5 Players)')
plt.legend()
plt.tight_layout()
plt.show()

player_name = 'Roger Federer'
player_data = df[df['Player'] == player_name]

plt.figure(figsize=(8, 5))
plt.plot(player_data['Match'], player_data['WinPercentage'], marker='o', color='green')
plt.title(f'Win Percentage Trend - {player_name}')
plt.xlabel('Match Number')
plt.ylabel('Win Percentage')
plt.ylim(50, 105)
plt.grid(True)
plt.show()

num_cols = ['Aces', 'DoubleFaults', 'FirstServe%', 'BreakPointsSaved%', 'WinPercentage']
existing_cols = [col for col in num_cols if col in df.columns]

sns.pairplot(df[existing_cols], diag_kind='kde', plot_kws={'alpha': 0.6})
plt.suptitle('Performance Metrics Relationship', y=1.02)
plt.show()
```

```
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV> python ques_3.py
Dataset Loaded Successfully!
           Player  Match  Aces  DoubleFaults  FirstServe%  BreakPointsSaved%  WinPercentage
0   Novak Djokovic      1    12             2         68.4               80.2           95.1
1     Rafael Nadal      1     9             3         71.2               84.1           91.3
2    Roger Federer      1    11             4         69.5               78.5           88.6
3   Carlos Alcaraz      1    13             5         73.1               76.2           84.8
4  Daniil Medvedev      1    10             4         67.8               75.9           82.3
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV>
```

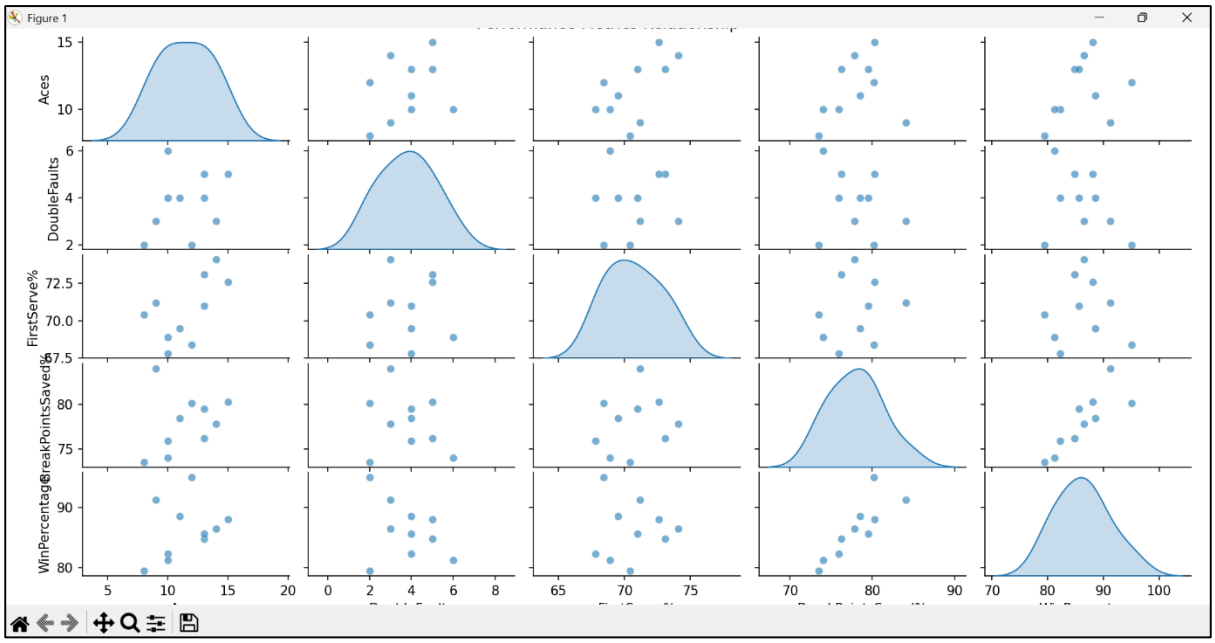
- Create a bar chart comparing the number of aces and double faults for top 5 players.



- Plot a line chart to show the win percentage trend of a player over a season.



- Use Seaborn's pairplot() function to explore the relationship between different performance metrics.



Ques 4.Patient Health Records Visualization:
- Given a dataset of patient health records (e.g., age, blood pressure, cholesterol levels):
- Create a box plot to identify outliers in blood pressure readings.
- Use a scatter plot to visualize the relationship between age and cholesterol levels.
- Plot the distribution of patients' ages using a histogram.

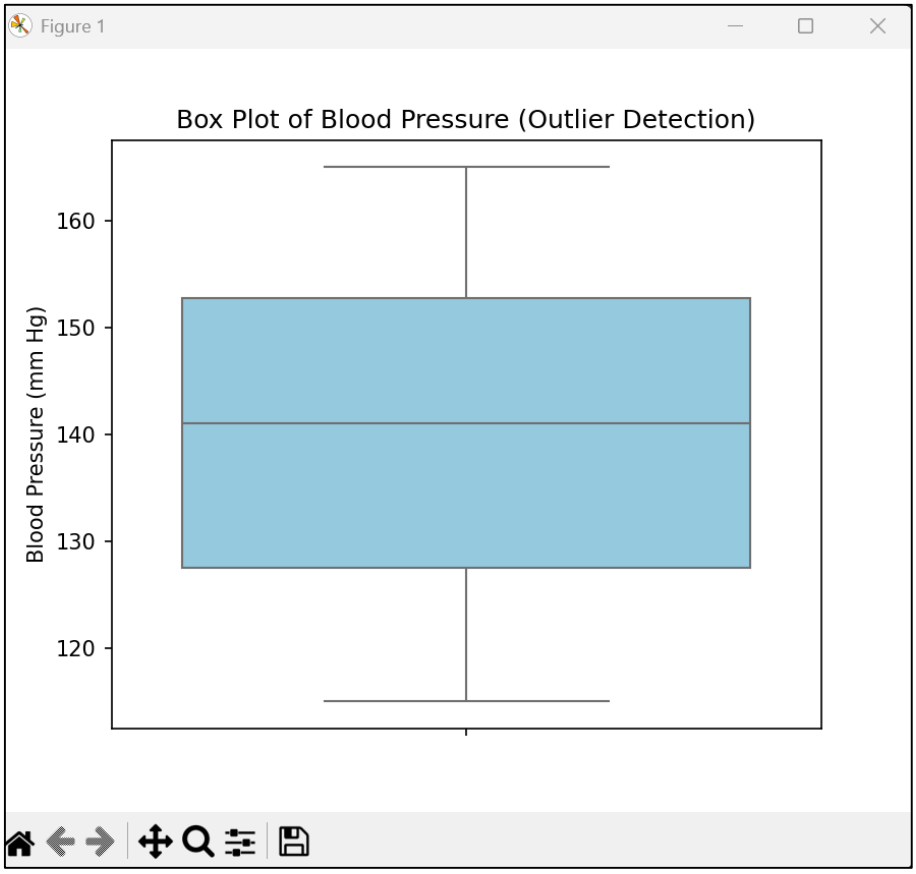
Code:-

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("patient_health.csv")

print("Dataset Loaded Successfully!")
print(df.head())

plt.figure(figsize=(6, 5))
sns.boxplot(y=df["BloodPressure"], color="skyblue")
plt.title("Box Plot of Blood Pressure (Outlier Detection)")
plt.ylabel("Blood Pressure (mm Hg)")
plt.show()

plt.figure(figsize=(7, 5))
```

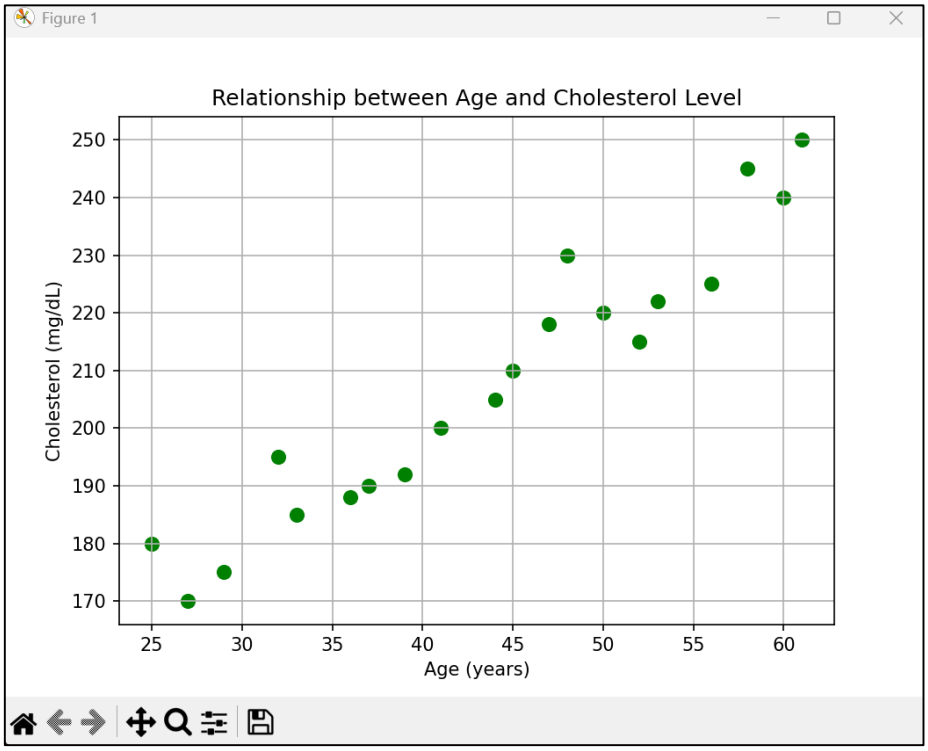
```
sns.scatterplot(x="Age", y="Cholesterol", data=df, color="green", s=80)
plt.title("Relationship between Age and Cholesterol Level")
plt.xlabel("Age (years)")
plt.ylabel("Cholesterol (mg/dL)")
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 5))
plt.hist(df["Age"], bins=8, color="salmon", edgecolor="black")
plt.title("Distribution of Patient Ages")
plt.xlabel("Age (years)")
plt.ylabel("Number of Patients")
plt.grid(axis='y')
plt.show()
```

```
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV> python ques_4.py
Dataset Loaded Successfully!
  PatientID  Age  BloodPressure  Cholesterol
0      P001   25           120          180
1      P002   32           130          195
2      P003   45           140          210
3      P004   29           118          175
4      P005   50           155          220
```

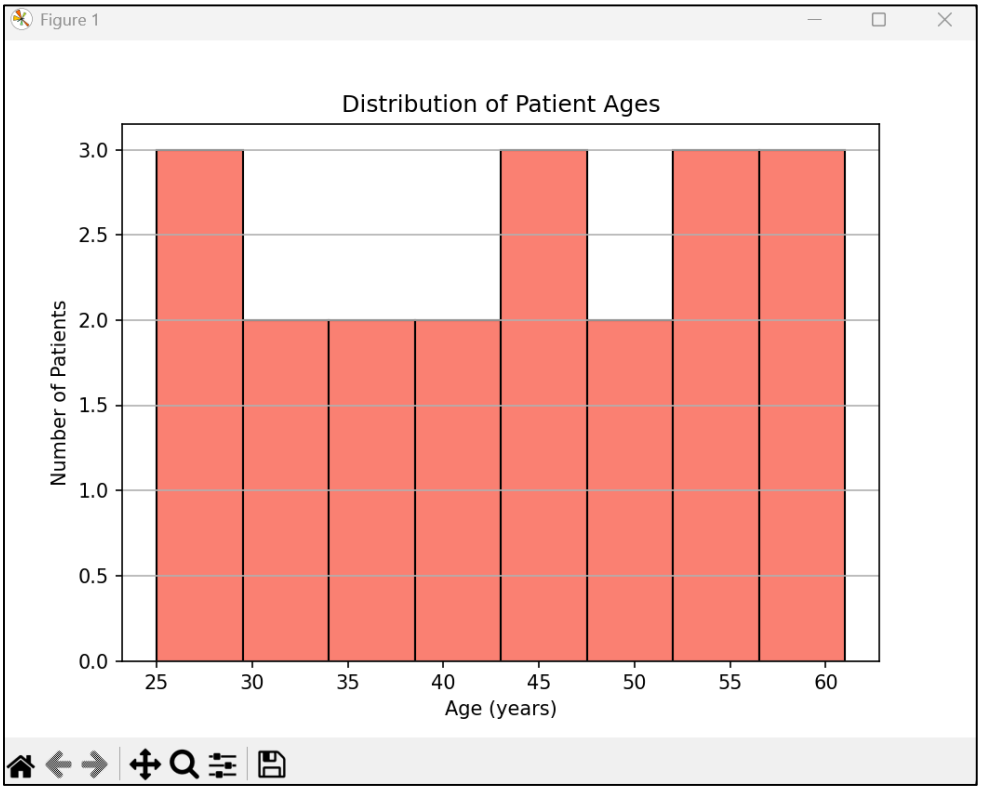
- Create a box plot to identify outliers in blood pressure readings.



- Use a scatter plot to visualize the relationship between age and cholesterol levels.



- Plot the distribution of patients' ages using a histogram.



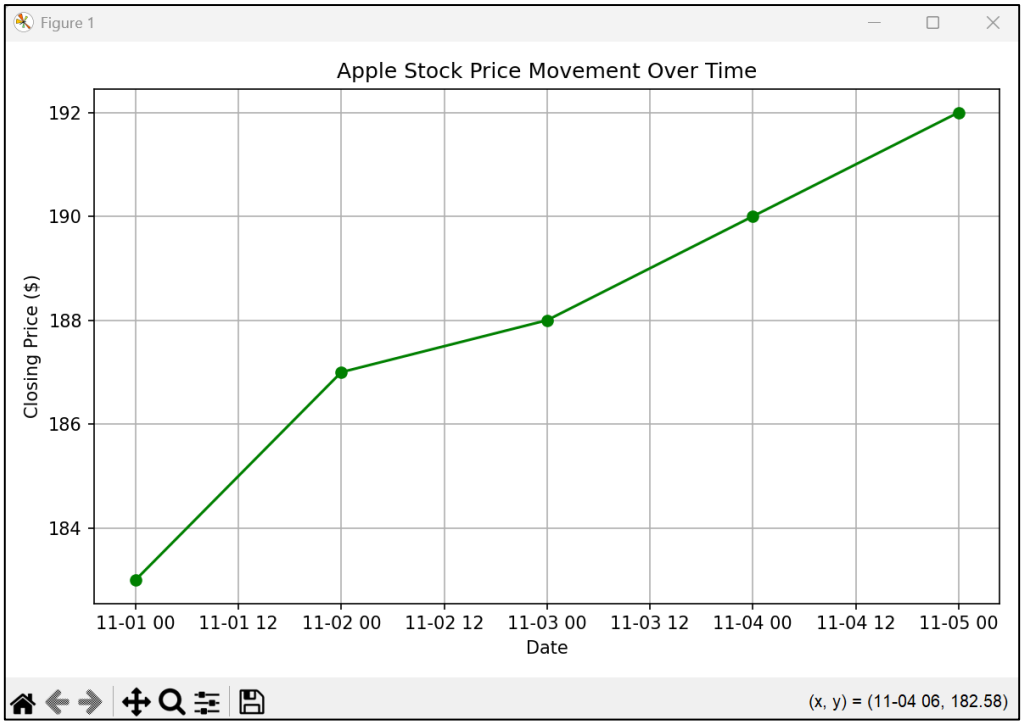
Ques 5. Stock Market Analysis:
- Load stock price data for multiple companies (e.g., open, close, high, low prices).
- Create a line chart to visualize the stock price movement of a company over time.
- Use a candlestick chart to show daily price fluctuations.
- Plot a correlation heatmap to examine the relationship between stock prices of different companies.

Code:-

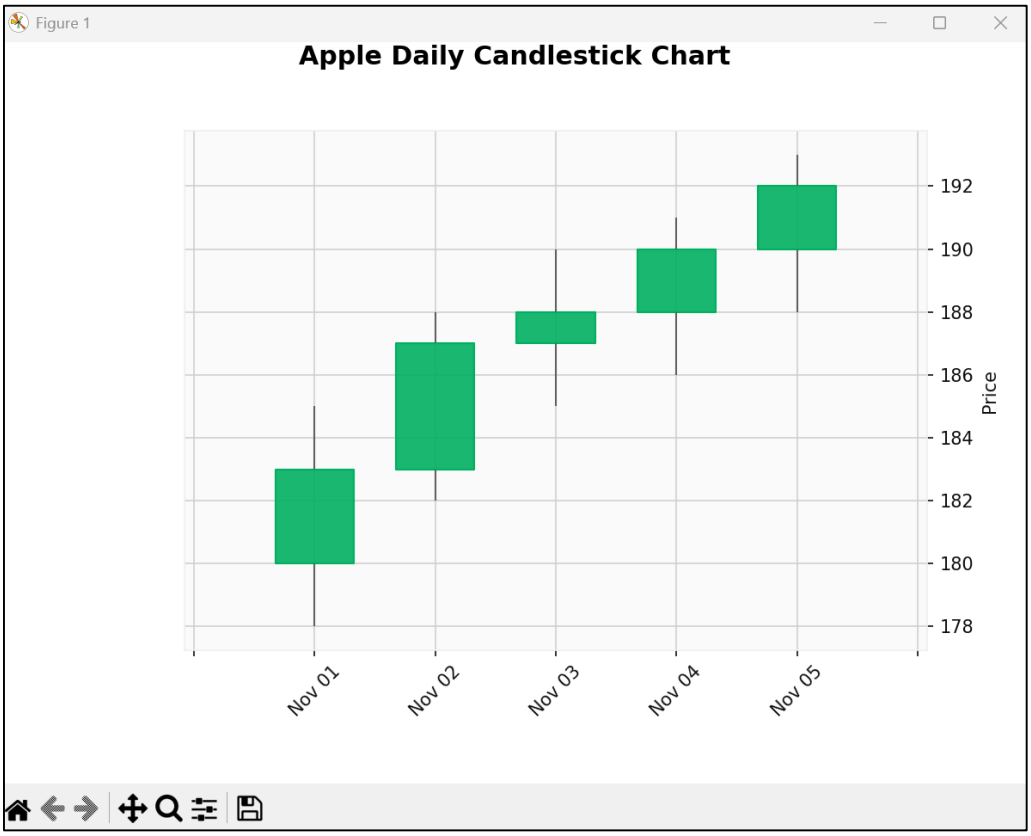
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import mplfinance as mpf  #pip install pandas matplotlib seaborn mplfinance

df = pd.read_csv("stock_data.csv")
df['Date'] = pd.to_datetime(df['Date'])
print("Data Loaded Successfully!")
print(df.head())

company = "Apple"
company_data = df[df['Company'] == company]

plt.figure(figsize=(8, 5))
plt.plot(company_data['Date'], company_data['Close'], marker='o', color='green')
plt.title(f"{company} Stock Price Movement Over Time")
plt.xlabel("Date")
plt.ylabel("Closing Price ($)")
plt.grid(True)
plt.tight_layout()
plt.show()

candlestick_data = company_data.set_index('Date')[['Open', 'High', 'Low', 'Close']]
mpf.plot(candlestick_data, type='candle', style='yahoo', title=f"{company} Daily Candlestick Chart")

pivot_df = df.pivot(index='Date', columns='Company', values='Close')

plt.figure(figsize=(7, 5))
sns.heatmap(pivot_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Between Companies' Closing Prices")
plt.tight_layout()
plt.show()
```

```
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV> python ques_5.py
Data Loaded Successfully!
        Date    Company  Open  High   Low  Close
0 2025-11-01      Apple   180   185   178    183
1 2025-11-01     Google  2750  2800  2720   2785
2 2025-11-01     Amazon  3350  3400  3300   3385
3 2025-11-01  Microsoft   350   355   348    352
4 2025-11-02      Apple   183   188   182    187
```

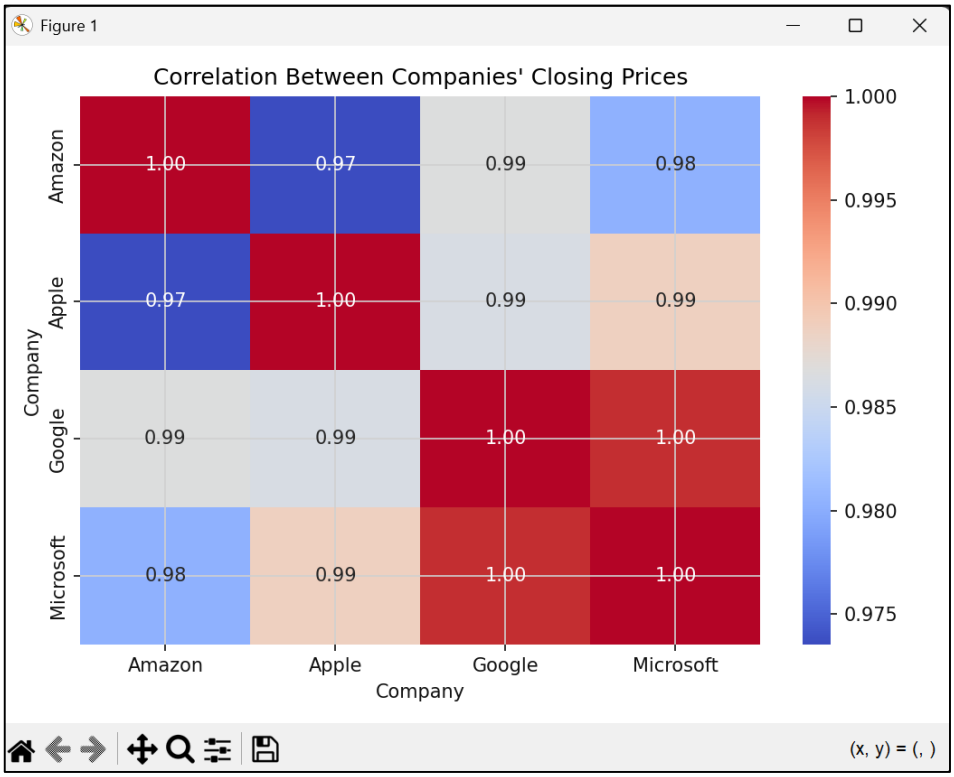
- Create a line chart to visualize the stock price movement of a company over time.



- Use a candlestick chart to show daily price fluctuations.



- Plot a correlation heatmap to examine the relationship between stock prices of different companies.



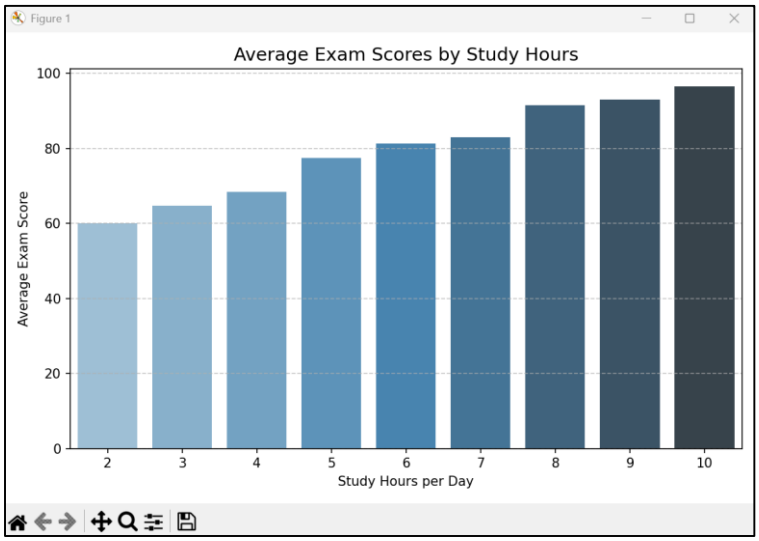
6. Student Performance Analysis:
- Load a dataset containing student performance (e.g., exam scores, attendance, study hours).
- Create a bar chart to compare the average exam scores for students across different study hours.
- Use a box plot to visualize the distribution of exam scores across different grades.
- Plot a scatter plot to show the relationship between study hours and exam scores.

Code:-

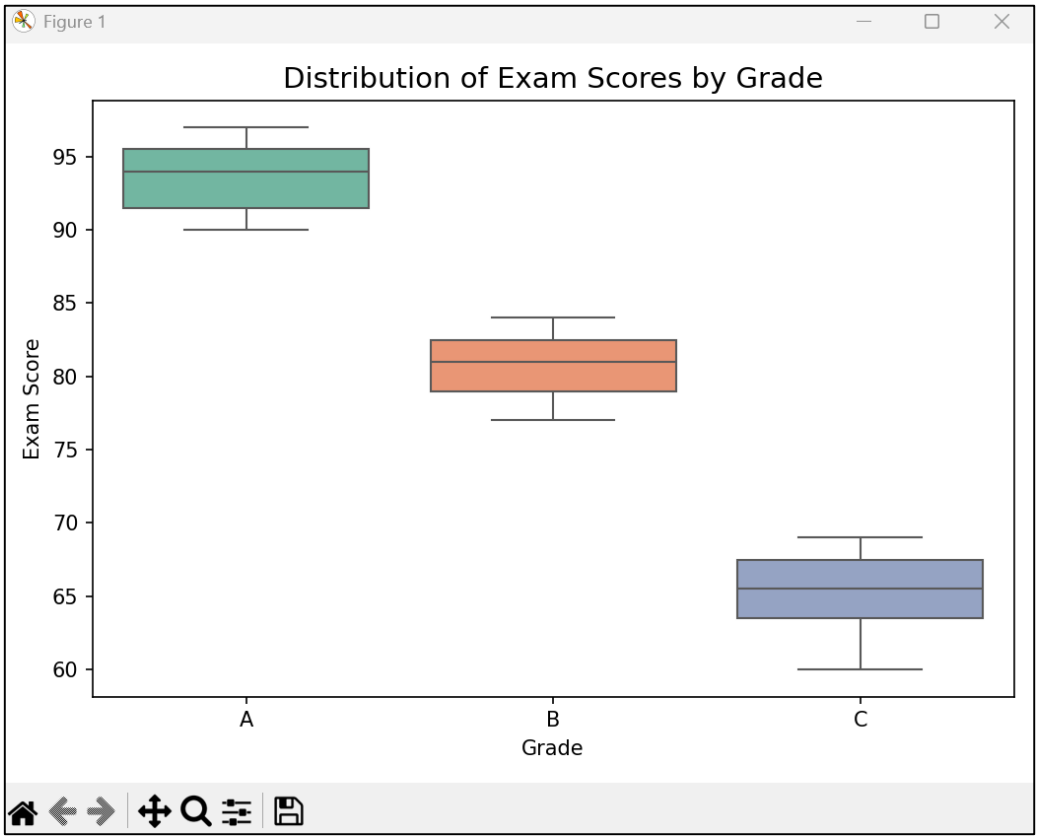
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("student_performance.csv")
print("Data Loaded Successfully!")
print(df.head())

avg_scores = df.groupby("StudyHours")["ExamScore"].mean().reset_index()

plt.figure(figsize=(8, 5))
sns.barplot(x="StudyHours", y="ExamScore", data=avg_scores, palette="Blues_d")
plt.title("Average Exam Scores by Study Hours", fontsize=14)
plt.xlabel("Study Hours per Day")
plt.ylabel("Average Exam Score")
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 5))
sns.boxplot(x="Grade", y="ExamScore", data=df, palette="Set2")
plt.title("Distribution of Exam Scores by Grade", fontsize=14)
plt.xlabel("Grade")
plt.ylabel("Exam Score")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 5))
sns.scatterplot(x="StudyHours", y="ExamScore", data=df, hue="Grade", s=80, palette="coolwarm")
plt.title("Relationship Between Study Hours and Exam Scores", fontsize=14)
plt.xlabel("Study Hours per Day")
plt.ylabel("Exam Score")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV> python ques_6.py
Data Loaded Successfully!
  StudentID Grade  StudyHours  Attendance  ExamScore
0      S001     A           8          95         92
1      S002     B           6          88         80
2      S003     A           9          97         95
3      S004     C           3          70         65
4      S005     B           5          85         78
```

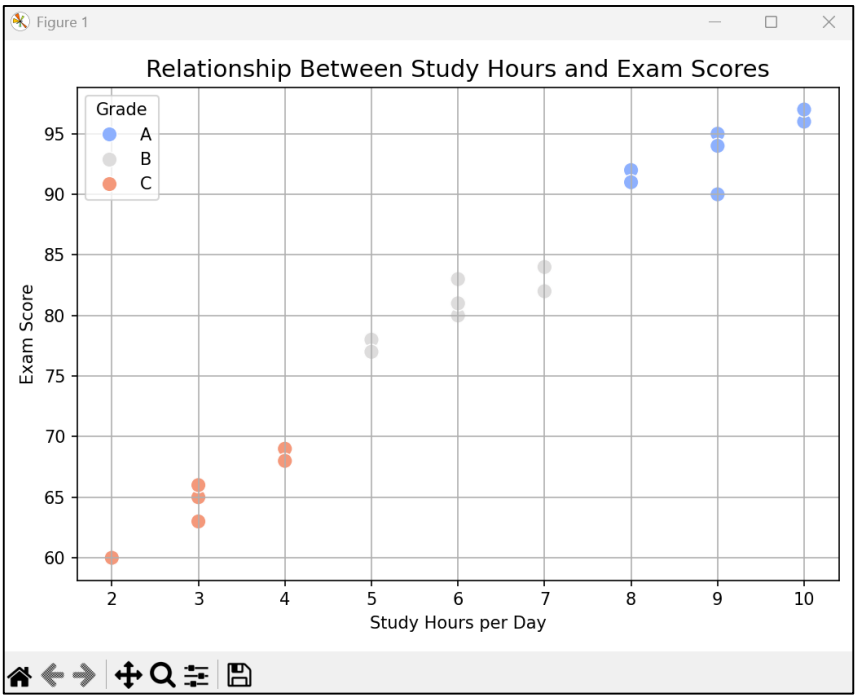
- Create a bar chart to compare the average exam scores for students across different study hours.



- Use a box plot to visualize the distribution of exam scores across different grades.



- Plot a scatter plot to show the relationship between study hours and exam scores.



7.University Admissions Data:

- Use a dataset containing university admissions data (e.g., GPA, SAT scores, admission

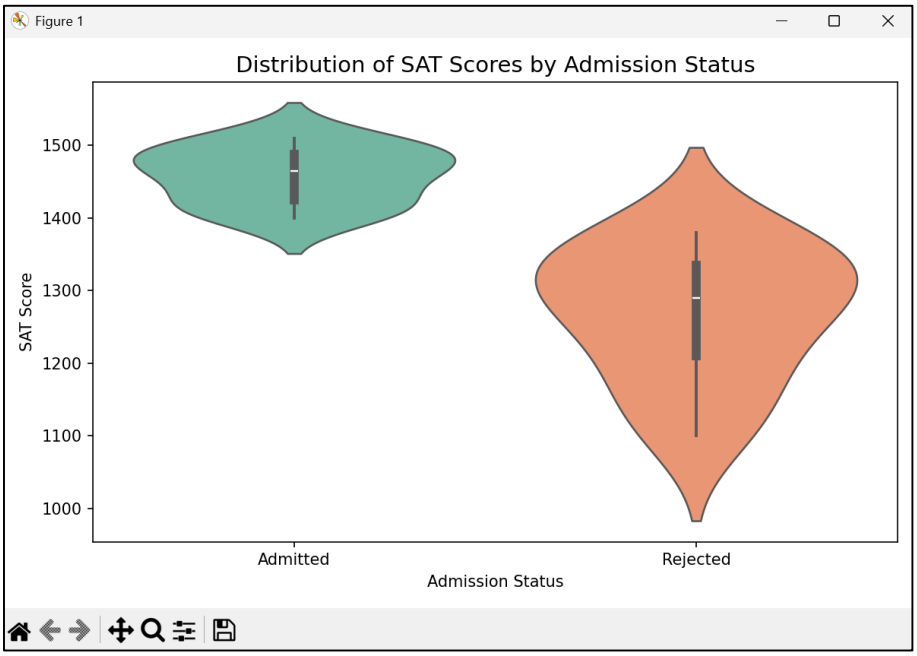
status).
- Create a violin plot to show the distribution of SAT scores for admitted and rejected students.
- Use a histogram to visualize the distribution of GPA scores.
- Plot a scatter plot to explore the relationship between GPA and SAT scores for admitted students.

Code:-

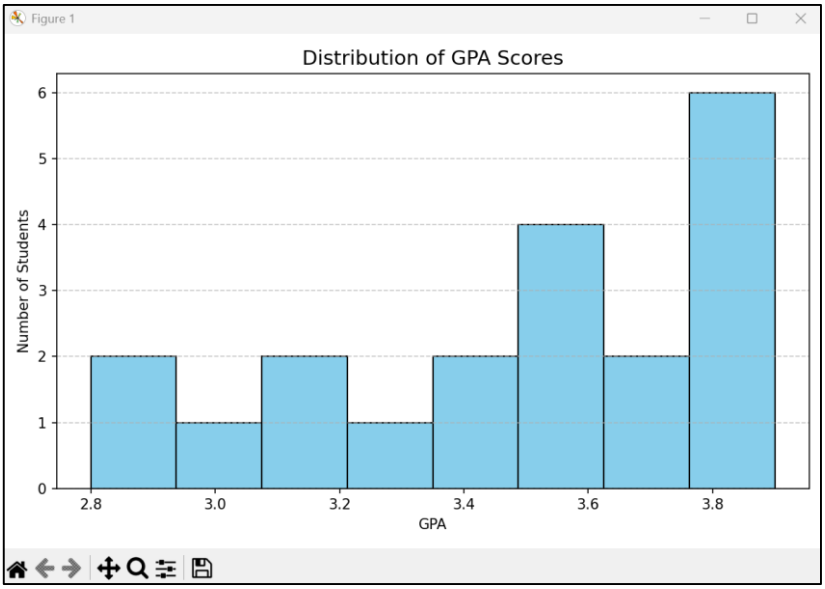
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("university_admissions.csv")
print(" Data Loaded Successfully!")
print(df.head())

plt.figure(figsize=(8, 5))
sns.violinplot(x="AdmissionStatus", y="SAT", data=df, palette="Set2")
plt.title("Distribution of SAT Scores by Admission Status", fontsize=14)
plt.xlabel("Admission Status")
plt.ylabel("SAT Score")
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(df["GPA"], bins=8, color="skyblue", edgecolor="black")
plt.title("Distribution of GPA Scores", fontsize=14)
plt.xlabel("GPA")
plt.ylabel("Number of Students")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

admitted_df = df[df["AdmissionStatus"] == "Admitted"]

plt.figure(figsize=(8, 5))
sns.scatterplot(x="GPA", y="SAT", data=admitted_df, color="green", s=80)
plt.title("GPA vs SAT Scores (Admitted Students)", fontsize=14)
plt.xlabel("GPA")
plt.ylabel("SAT Score")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\HP\Desktop\MSC-CA SEM-3\Data intelligence & visualizations\DIV> python ques_7.py
 Data Loaded Successfully!
  StudentID  GPA   SAT AdmissionStatus
0      S001  3.9  1500        Admitted
1      S002  3.7  1420        Admitted
2      S003  3.5  1380        Rejected
3      S004  3.2  1280        Rejected
4      S005  3.8  1460        Admitted
```

- Create a violin plot to show the distribution of SAT scores for admitted and rejected students.



- Use a histogram to visualize the distribution of GPA scores.



- Plot a scatter plot to explore the relationship between GPA and SAT scores for admitted students.